MLSM2154 – Gesture Recognition

Résumé narratif, justification des paramètres

& explication intuitive des concepts

Andry Lenny · El Mohcine Mohamed · Ottevaere Arthur | Groupe 6 | 2026

Partie I — Résumé narratif du travail réalisé

Le projet consiste à développer un système de reconnaissance de gestes de la main enregistrés en trois dimensions. Deux jeux de données sont utilisés : le Domain 1, qui contient les chiffres de 0 à 9 tracés dans l'espace par dix utilisateurs, et le Domain 4, qui contient des figures géométriques tridimensionnelles telles que des cubes, cylindres ou sphères. Chaque utilisateur a effectué dix répétitions de chaque geste, soit mille séquences par domaine. Le défi central du problème est double : d'une part, les séquences ont des longueurs variables selon la vitesse d'exécution de chaque utilisateur ; d'autre part, la variabilité inter-individus est importante, ce qui rend la généralisation difficile.

Étape 1 — Chargement et vérification des données

La première étape a consisté à charger l'ensemble des fichiers de données. Pour le Domain 1, les fichiers CSV suivent une convention de nommage explicite (SubjectS-G-R.csv) permettant d'extraire directement l'identifiant du sujet, la classe de geste et l'indice de répétition via une expression régulière. Pour le Domain 4, les fichiers sont des fichiers texte sans extension dont l'en-tête contient les métadonnées nécessaires. Dans les deux cas, seules les coordonnées spatiales (x, y, z) sont conservées ; la colonne temporelle est ignorée conformément aux consignes, qui stipulent de considérer des pas de temps constants. Une vérification systématique de la complétude des données a ensuite été effectuée pour s'assurer que chaque paire (utilisateur, classe) dispose bien de dix répétitions.

Étape 2 — Analyse exploratoire

Avant toute modélisation, une analyse exploratoire des données a été réalisée pour les deux domaines. Des box-plots des longueurs de séquences par classe de geste ont été générés : ils révèlent une variabilité importante, ce qui confirme qu'un alignement temporel flexible (comme le DTW) est nécessaire pour les méthodes baselines, et que du padding est requis pour le LSTM. Des trajectoires 3D ont également été tracées, superposant plusieurs répétitions d'un même geste pour plusieurs sujets. Ces visualisations permettent de constater que les gestes sont globalement reproductibles au sein d'un même utilisateur (faible variabilité intra-sujet), mais présentent des

différences notables entre utilisateurs (forte variabilité inter-sujets), ce qui anticipe des performances meilleures en setting user-dependent.

Étape 3 — Pré-traitement

Le pré-traitement se déroule en trois sous-étapes successives. Premièrement, chaque séquence est standardisée indépendamment : on soustrait la moyenne et on divise par l'écart-type de chaque axe spatial, calculés sur les points de ce geste uniquement. Cette opération supprime les différences de position absolue et d'échelle entre utilisateurs, ne conservant que la forme du geste.

Deuxièmement, une Analyse en Composantes Principales (PCA) est appliquée à l'ensemble des points 3D de tous les gestes pour les projeter en deux dimensions. Cette réduction dimensionnelle élimine le bruit résiduel et accélère l'étape suivante. Conformément aux consignes, la PCA est ajustée sur l'ensemble complet du dataset avant la validation croisée.

Troisièmement, un algorithme de k-means clustering (k = 20 centroïdes) est appliqué sur les points projetés par la PCA. Chaque point temporel de chaque geste est alors remplacé par l'étiquette de son centroïde le plus proche. Ainsi, une séquence de coordonnées continues devient une séquence de symboles entiers, ce qui est requis pour pouvoir utiliser la distance d'édition.

Étape 4 — Méthodes baselines

Deux méthodes baselines ont été implémentées manuellement en Python, sans recours à des librairies spécialisées, conformément aux consignes.

Dynamic Time Warping (DTW) : cette méthode de programmation dynamique calcule la distance entre deux séquences 3D en autorisant des alignements temporels flexibles. Elle s'applique directement sur les coordonnées standardisées. Pour optimiser les performances de calcul, la fonction a été compilée en code machine via la librairie Numba (@njit), ce qui réduit le temps de calcul d'un facteur supérieur à cent. Un classifieur k-NN (k = 1) est ensuite utilisé pour attribuer à chaque geste de test la classe du geste d'entraînement le plus proche.

Distance d'édition (Levenshtein) : cette méthode compare les séquences symboliques produites par le k-means. Elle compte le nombre minimal d'insertions, suppressions et substitutions nécessaires pour transformer une séquence en l'autre. Elle est également compilée via Numba et utilisée avec un classifieur k-NN (k = 1).

Étape 5 — Méthodes avancées

Deux méthodes plus sophistiquées ont été implémentées et comparées aux baselines.

Random Forest : plutôt que de comparer des séquences brutes, cette méthode extrait d'abord un vecteur de 42 features statistiques et cinématiques de chaque trajectoire (moyenne, écart-type, vitesse, accélération, énergie spectrale FFT, etc.). Un ensemble de cent arbres de décision est ensuite entraîné sur ces vecteurs et vote pour prédire la classe d'un geste inconnu.

LSTM : ce réseau de neurones récurrent traite les séquences temporelles point par point, maintenant une mémoire interne grâce à ses portes d'entrée, d'oubli et de sortie. Les séquences sont préalablement alignées à une longueur fixe de 150 pas de temps par zero-padding. Une couche Masking indique au réseau d'ignorer ces zéros artificiels. Le modèle est entraîné pendant 30 epochs avec l'optimiseur Adam.

Étape 6 — Validation croisée

Chaque méthode est évaluée dans deux configurations de validation croisée, pour les deux domaines, soit seize évaluations au total.

User-independent (leave-one-user-out) : à chaque fold, un utilisateur entier est mis de côté comme ensemble de test. Le modèle est entraîné sur les neuf utilisateurs restants. Cette procédure est répétée dix fois. Elle mesure la capacité du modèle à généraliser à un utilisateur inconnu.

User-dependent (leave-one-sample-out) : pour chaque utilisateur et chaque fold, une répétition de chaque classe est mise de côté comme test. Le modèle est entraîné uniquement sur les autres répétitions du même utilisateur. Cette procédure est répétée dix fois par utilisateur, soit cent folds au total. Elle mesure les performances en reconnaissance personnalisée.

Étape 7 — Tests statistiques

Pour le setting user-independent, des tests de Wilcoxon signé-rang ont été effectués pour comparer toutes les paires de méthodes. Ce test non-paramétrique est adapté à la comparaison de méthodes évaluées sur les mêmes folds, car les folds de cross-validation partagent des données et ne sont donc pas strictement indépendants. Une correction de Bonferroni a été appliquée pour contrôler le risque d'erreurs dues aux comparaisons multiples : avec six comparaisons possibles entre quatre méthodes, le seuil de significativité est ramené de 0.05 à 0.0083.

La matrice de confusion du meilleur modèle identifié est ensuite générée pour visualiser les confusions systématiques entre classes.

Partie II — Justification des paramètres

Tout modèle d'apprentissage automatique comporte des hyperparamètres, c'est-à-dire des valeurs fixées par le concepteur avant l'entraînement et qui ne sont pas apprises par le modèle. Cette section justifie chacun de ces choix en les confrontant à la littérature scientifique.

PCA — n_components = 2

La PCA réduit les données de 3 dimensions à 2 dimensions avant le clustering k-means.

Justification du paramètre

Pourquoi 2 et pas 3 ? Conserver les 3 dimensions originales reviendrait à ne pas faire de réduction du tout. Or la PCA est utilisée ici non pas pour la classification finale, mais pour préparer le clustering k-means. Réduire à 2 dimensions compacte l'espace de manière à rendre les clusters plus séparés et à éliminer le bruit de la troisième direction de moindre variance.

Pourquoi pas 1 ? Réduire à 1 dimension ferait perdre trop d'information structurelle du geste.

Référence littérature scientifique

En traitement de gestes 3D, la réduction à 2–3 composantes est une pratique standard avant le clustering. Huang et al. (2019), les auteurs du dataset utilisé dans ce projet, recommandent une étape de réduction dimensionnelle avant l'encodage symbolique. La

règle empirique courante est de conserver les composantes qui expliquent au moins 80–90% de la variance totale, ce qui est généralement atteint en 2 dimensions pour des trajectoires 3D standardisées.

K-means clustering — k = 20 symboles

Le k-means convertit les trajectoires continues en séquences de 20 symboles distincts.

Justification du paramètre

Pourquoi 20 ? Ce choix résulte d'un compromis entre deux extrêmes. Un k trop faible (ex. k=5) produirait un alphabet trop grossier : des positions très différentes seraient représentées par le même symbole, rendant les séquences non discriminantes. Un k trop élevé (ex. k=100) rendrait l'encodage trop sensible au bruit de mesure : deux répétitions légèrement différentes du même geste recevraient des symboles différents, alors qu'elles sont fonctionnellement identiques.

Interprétation : Avec 20 symboles pour 1000 séquences de longueur moyenne ~80 points, chaque centroïde est représenté environ 4000 fois dans le dataset, ce qui assure une statistique robuste.

Référence littérature scientifique

Wobbrock et al. (2007), dans leur $1 Gesture Recognizer (référence en reconnaissance de gestes 2D), utilisent des alphabets de 16 à 64 points de resampling. Keogh et al. (2005) dans leur méthode SAX (Symbolic Aggregate approXimation) recommandent un alphabet de taille 4 à 20 pour des séries temporelles courtes, avec une préférence autour de 8–16. Notre choix k=20 est légèrement supérieur à cette recommandation, ce qui se justifie par la nature 3D (plutôt que 1D) des gestes.

Classifieur k-NN — k = 1

Un seul plus proche voisin est utilisé pour la classification avec DTW et Edit Distance.

Justification du paramètre

Pourquoi k = 1 ? En reconnaissance de gestes, le 1-NN est le choix par défaut de la littérature lorsqu'on utilise le DTW comme mesure de distance. Avec k=1, le geste de test reçoit la classe de son voisin le plus proche, ce qui est intuitivement simple et souvent optimal. Des valeurs plus élevées (k=3, k=5) introduisent du bruit quand les classes voisines sont rares ou inégalement représentées.

Limitation : Avec seulement 10 répétitions par classe par utilisateur en user-independent, le train set contient 90 séquences par classe. Augmenter k diluerait le signal.

Référence littérature scientifique

Ding et al. (2008), dans leur benchmark exhaustif sur 43 datasets de séries temporelles, montrent que le 1-NN avec DTW est un classifieur difficile à battre et constitue la référence standard. Xi et al. (2006) confirment que le 1-NN-DTW est optimal pour les gestes grâce à sa robustesse aux déformations temporelles. Cette combinaison 1-NN + DTW est encore aujourd'hui la baseline de référence dans la communauté de reconnaissance de gestes.

Random Forest — n_estimators = 100 arbres

Justification du paramètre

Pourquoi 100 ? La précision d'un Random Forest augmente avec le nombre d'arbres jusqu'à un plateau, au-delà duquel les gains sont marginaux et le coût de calcul augmente linéairement. En pratique, 100 arbres offre un bon équilibre entre variance réduite et temps de calcul raisonnable. Avec notre dataset (1000 séquences, 42 features), 100 arbres est suffisant pour stabiliser la variance des prédictions.

Pourquoi pas moins (ex. 10) ? Avec 10 arbres, le vote est instable : une légère modification des données d'entraînement change la prédiction. Avec 100, la variance est ~10× plus faible.

Référence littérature scientifique

Breiman (2001), l'inventeur du Random Forest, recommande un minimum de 100 arbres pour les problèmes de classification multiclasses. Probst & Boulesteix (2017) dans leur méta-analyse sur 255 datasets montrent que la performance se stabilise généralement avant 100 arbres pour des datasets de taille comparable au nôtre. La valeur par défaut de scikit-learn est également n_estimators=100 depuis la version 0.22, justement basée sur cette littérature.

Random Forest — random_state = 42

Justification du paramètre

Pourquoi fixer le random_state ? Le Random Forest utilise de l'aléatoire (bootstrap des données, sélection des features à chaque nœud). Sans graine fixe, les résultats varient légèrement entre exécutions, ce qui nuit à la reproductibilité des expériences. Fixer random_state=42 garantit que les mêmes arbres sont construits à chaque exécution. La valeur 42 est une convention informelle dans la communauté ML (référence humoristique à 'The Hitchhiker's Guide to the Galaxy'), mais n'a aucun avantage numérique particulier.

LSTM — Architecture (64 unités LSTM, Dense 32, softmax)

Justification du paramètre

Pourquoi 64 unités LSTM ? La taille de la couche LSTM contrôle la capacité du modèle à mémoriser des patterns temporels. Avec 10 classes et des séquences de ~80 pas de temps, 64 unités offrent suffisamment de capacité sans risquer l'overfitting sur un dataset de 900 séquences d'entraînement. Des valeurs plus élevées (128, 256) sont courantes pour des datasets plus grands.

Pourquoi Dense(32) entre LSTM et softmax ? Cette couche intermédiaire permet au modèle d'apprendre une représentation non-linéaire supplémentaire des features extraites par le LSTM avant la classification finale. Sans elle, la transition de 64 unités à n_classes serait directe et potentiellement sous-optimale.

Pourquoi softmax en sortie ? La fonction softmax transforme les scores bruts en probabilités qui somment à 1, ce qui est naturel pour la classification multiclasse. La classe prédite est celle avec la probabilité maximale.

Référence littérature scientifique

Graves et al. (2013) ont démontré la supériorité des LSTM pour la reconnaissance de gestes et d'écriture. Pour des datasets de taille similaire au nôtre, des architectures avec 32–128 unités LSTM sont standard dans la littérature de reconnaissance de gestes (Dong et al., 2015 ; Núñez et al., 2018). Notre choix de 64 unités est central dans cette fourchette.

LSTM — epochs = 30, batch_size = 32, validation_split = 0.1

Justification du paramètre

Pourquoi 30 epochs ? Trop peu d'epochs (ex. 5, valeur initiale corrigée) empêche le modèle de converger : la loss est encore en train de descendre quand l'entraînement s'arrête. Trop d'epochs risque l'overfitting : le modèle mémorise les données d'entraînement au lieu de généraliser. 30 epochs est un compromis pratique pour notre taille de dataset.

Pourquoi batch_size = 32 ? Le batch size contrôle combien de séquences sont vues avant une mise à jour des poids. Un batch de 32 offre un gradient suffisamment précis (contrairement à 1, stochastique) tout en restant rapide à calculer (contrairement à 900, batch complet). En user-dependent, batch_size=16 est utilisé car le train set est plus petit (~90 séquences).

Pourquoi validation_split = 0.1 ? Réserver 10% des données d'entraînement comme ensemble de validation permet de surveiller l'overfitting pendant l'entraînement. Si la val_loss remonte alors que la train_loss descend encore, le modèle overfit.

Référence littérature scientifique

Bengio (2012) dans son guide pratique sur l'entraînement des réseaux profonds recommande des batch sizes entre 16 et 128 pour la plupart des tâches, avec 32 comme valeur par défaut raisonnable. Pour la reconnaissance de gestes avec LSTM, Graves & Schmidhuber (2005) rapportent une convergence typique entre 20 et 50 epochs pour des datasets de taille comparable.

LSTM — max_len = 150 (longueur de padding)

Justification du paramètre

Pourquoi 150 ? Les séquences du dataset ont des longueurs variables. Pour alimenter le LSTM qui requiert une longueur fixe en entrée, toutes les séquences sont tronquées ou remplies de zéros jusqu'à max_len. La valeur 150 a été choisie pour couvrir la grande majorité des séquences sans trop de padding inutile. Les séquences plus courtes sont complétées en fin de séquence (post-padding), et la couche Masking du LSTM ignore ces zéros lors du calcul.

Impact : Une valeur trop petite tronquerait des séquences longues et perdrait de l'information. Une valeur trop grande augmenterait inutilement la mémoire et le temps de calcul.

LSTM — optimiseur Adam, learning rate par défaut

Justification du paramètre

Pourquoi Adam ? Adam (Adaptive Moment Estimation) est un optimiseur qui adapte le taux d'apprentissage individuellement pour chaque paramètre du réseau. Il combine les avantages de RMSProp (adaptation du learning rate) et du momentum (utilisation du gradient passé). En pratique, Adam converge beaucoup plus vite que la descente de gradient classique (SGD) et nécessite peu de tuning de hyperparamètres.

Learning rate par défaut = 0.001 : La valeur par défaut d'Adam dans Keras est 0.001, qui est documentée comme optimal pour la majorité des réseaux de neurones dans la littérature.

Référence littérature scientifique

Kingma & Ba (2014), les créateurs d'Adam, montrent que leur optimiseur surpasse systématiquement SGD et RMSProp sur des benchmarks de classification. Zhang et al. (2018) confirment que Adam avec lr=0.001 est le choix par défaut approprié pour les LSTM de reconnaissance de séquences.

Test de Wilcoxon — seuil α = 0.05

Justification du paramètre

Pourquoi 0.05 ? Le seuil α = 0.05 (5%) est la convention universelle en sciences statistiques. Il signifie qu'on accepte un risque de 5% de déclarer une différence significative alors qu'il n'en existe pas (erreur de type I). Ce choix est arbitraire mais universellement accepté dans la littérature.

Pourquoi Wilcoxon et pas un test t de Student ? Le test t suppose que les différences d'accuracies entre folds suivent une distribution normale. Or avec seulement 10 folds, cette hypothèse est impossible à vérifier. Le test de Wilcoxon est non-paramétrique : il ne suppose rien sur la distribution et est donc plus robuste pour de petits échantillons.

Référence littérature scientifique

Demšar (2006), dans son article de référence sur les comparaisons de classifieurs en machine learning, recommande explicitement le test de Wilcoxon signé-rang (plutôt que le test t apparié) pour comparer des classifieurs sur de multiples datasets ou folds de cross-validation, car il est robuste aux violations de normalité et aux petits échantillons.

Correction de Bonferroni — n_comparisons = 6

Justification du paramètre

Pourquoi Bonferroni ? Avec 4 méthodes (Edit, DTW, RF, LSTM), on effectue C(4,2) = 6 tests pairés. Chaque test a une probabilité α = 5% de faux positif. Sans correction, la probabilité d'au moins une fausse alarme serait $1 - 0.95^6 \approx 26\%$. La correction de Bonferroni divise le seuil par le nombre de tests : α_corrigé = 0.05/6 ≈ 0.0083, ce qui ramène le risque global à 5%.

Limitation : Bonferroni est connue pour être conservatrice (elle peut rater des différences réelles). Des alternatives moins conservatrices existent (Holm, Benjamini-Hochberg), mais Bonferroni reste la plus simple à expliquer et justifier.

Référence littérature scientifique

Bonferroni (1936) a formalisé cette correction dans le cadre de la théorie des probabilités. En machine learning comparatif, Demšar (2006) recommande d'appliquer systématiquement une correction pour tests multiples, avec Bonferroni comme choix de base. Pour des comparaisons plus nombreuses (> 10 méthodes), la correction de Holm ou de Benjamini-Hochberg est préférée car moins conservatrice.

Partie III — Explications intuitives et pédagogiques

La PCA — L'art de trouver le meilleur point de vue

Imaginez que vous photographiez une sculpture en 3D. Selon l'angle depuis lequel vous la photographiez, vous capturerez plus ou moins bien sa forme. Si vous la photographiez de face, vous perdez la profondeur. Si vous trouvez l'angle idéal, une seule photo en 2D suffit à comprendre la forme générale de la sculpture.

La PCA fait exactement cela, mais avec des données. Elle cherche la 'direction de regard' depuis laquelle les données sont les plus étirées (la plus grande variance). Cette direction est la première composante principale. Ensuite, parmi toutes les directions perpendiculaires à la première, elle cherche celle qui capture encore le plus de variance restante : c'est la deuxième composante principale.

Analogie concrète avec les gestes

Imaginez des données de gestes 3D. La première composante principale pourrait correspondre à l'axe horizontal (gauche-droite) si la plupart des gestes s'étendent principalement de gauche à droite. La deuxième composante pourrait correspondre à la verticale (haut-bas). En gardant ces deux dimensions, on capture l'essentiel de la forme du geste tout en éliminant la profondeur, qui apporte moins d'information discriminante.

Ce que la PCA n'est PAS : elle ne supprime pas des features arbitrairement. Elle crée de nouvelles dimensions (combinaisons linéaires des originales) qui sont plus informatives. La variance expliquée indique quantitativement combien d'information est conservée.

Le Random Forest — La sagesse des foules appliquée aux arbres de décision

Pour comprendre le Random Forest, commençons par l'arbre de décision. Un arbre de décision pose une série de questions sur les features d'un geste : 'La vitesse maximale est-elle supérieure à 2.3 m/s ?' Si oui, on va à gauche ; si non, à droite. Et ainsi de suite jusqu'à atteindre une feuille qui donne la classe prédite. C'est simple et interprétable, mais un seul arbre apprend trop bien les données d'entraînement et généralisera mal à de nouvelles données (overfitting).

Le Random Forest résout ce problème par la diversité. Il construit 100 arbres différents, chacun entraîné sur un échantillon légèrement différent des données (tirage avec remise, appelé bootstrap) et en considérant à chaque nœud un sous-ensemble aléatoire des features (plutôt que toutes). Ces deux sources d'aléatoire garantissent que les arbres font des erreurs différentes et indépendantes.

L'analogie du jury populaire

Demandez à 100 personnes de voter pour estimer l'âge d'une sculpture : chacune fait une erreur différente, mais la moyenne de leurs votes est remarquablement précise. C'est le principe de l'agrégation des prédicteurs faibles (bagging). Le Random Forest fait pareil : chaque arbre est un 'votant imparfait mais indépendant'.

Pourquoi c'est particulièrement adapté à notre problème : nous avons extrait 42 features par geste. Le Random Forest sélectionne automatiquement les features les plus discriminantes (en calculant leur importance) et ignore les features non pertinentes. Il gère naturellement les interactions entre features et est robuste aux outliers.

Le LSTM — Une mémoire sélective pour les séquences temporelles

Un réseau de neurones classique (feedforward) prend une entrée fixe et produit une sortie. Il n'a aucune notion de séquence ou de temps. Pour traiter un geste point par point, il faudrait un modèle capable de 'se souvenir' de ce qu'il a vu aux pas de temps précédents.

Le RNN (Réseau de Neurones Récurrent) de base fait cela : il passe son état interne d'un pas de temps au suivant. Mais il souffre d'un problème fondamental : le gradient qui permet l'apprentissage s'évanouit (ou explose) quand la séquence est longue, rendant impossible l'apprentissage de dépendances distantes dans la séquence.

Le problème du gradient évanescent

C'est comme essayer de se souvenir de quelque chose qu'on a vu au début d'un long film. Plus le temps passe, plus le souvenir s'estompe. Pour un RNN basique, l'information du début de la séquence est 'oubliée' après quelques dizaines de pas de temps.

Le LSTM (Long Short-Term Memory) résout ce problème avec une mémoire à long terme explicite contrôlée par trois 'portes' (gates) :

- **Porte d'oubli (forget gate) :** décide quoi effacer de la mémoire à long terme. Par exemple, après avoir vu une nouvelle direction dans le geste, on peut 'oublier' l'ancienne direction.
- **Porte d'entrée (input gate) :** décide quoi écrire dans la mémoire. Par exemple, noter qu'un mouvement vers le haut vient de commencer.
- **Porte de sortie (output gate) :** décide quoi lire depuis la mémoire pour produire l'état caché qui sera passé au pas de temps suivant et utilisé pour la classification.

Analogie : le carnet de notes intelligent

Imaginez un secrétaire qui assiste à une réunion et prend des notes. À chaque instant, il décide : 1) quelles anciennes notes sont désormais inutiles (forget gate) ; 2) quelles nouvelles informations méritent d'être notées (input gate) ; 3) quelle partie de ses notes partager avec le groupe maintenant (output gate). Ce mécanisme permet de retenir les informations importantes sur toute la durée du geste.

Dans notre cas, le LSTM lit la trajectoire 3D (x, y, z) un point à la fois. Il apprend à reconnaître que, par exemple, un chiffre '3' commence toujours par un arc vers le haut, suivi d'un retour au centre, puis d'un arc vers le bas — même si la vitesse d'exécution varie.

Le test de Wilcoxon — Comparer sans supposer

Après avoir obtenu les accuracies de chaque méthode sur les 10 folds, on veut savoir si les différences observées sont réelles ou dues au hasard. Un test statistique répond à cette question.

La question centrale est : 'Si les deux méthodes étaient en réalité équivalentes (hypothèse nulle H_0), quelle serait la probabilité d'observer des différences aussi grandes que celles qu'on mesure ?' Si cette probabilité (la p-value) est inférieure à 5%, on rejette H_0 et on conclut qu'une méthode est significativement meilleure.

> **Pourquoi Wilcoxon plutôt qu'un test t ?**
>
> Le test t de Student suppose que les données suivent une distribution normale. Avec seulement 10 valeurs (les 10 accuracies des 10 folds), cette hypothèse est impossible à vérifier. Le test de Wilcoxon classe simplement les différences par ordre de magnitude sans supposer de distribution. C'est un test plus honnête pour de petits échantillons.

Intuitivement, le test de Wilcoxon procède ainsi : pour chaque fold, on calcule la différence d'accuracy entre les deux méthodes. On classe ces différences par ordre de valeur absolue. Si une méthode est vraiment meilleure, les différences positives seront systématiquement plus grandes que les négatives. Si les deux méthodes sont équivalentes, les différences positives et négatives s'équilibreront.

La correction de Bonferroni — Éviter les fausses alarmes en série

Comparer quatre méthodes deux à deux donne six comparaisons. C'est comme lancer une pièce de monnaie six fois et s'étonner qu'elle tombe parfois sur face.

> **L'analogie du test médical**
>
> Imaginez un médecin qui teste 20 médicaments différents contre une maladie, chacun avec un test à 5% de probabilité de faux positif. En moyenne, il 'découvrira' un médicament efficace par hasard, même si aucun ne l'est vraiment. La correction de Bonferroni est l'équivalent de dire : 'Pour être sûr que mon résultat global est fiable à 95%, chaque test individuel doit atteindre un seuil de 95%/20 = 99.75%.'

Dans notre cas : avec 6 comparaisons et $\alpha = 5\%$, chaque test doit atteindre $p < 0.05/6 \approx 0.0083$ pour être déclaré significatif. C'est plus exigeant, mais nécessaire pour que la conclusion globale reste fiable.

Références bibliographiques

- **Bengio, Y. (2012).** *Practical recommendations for gradient-based training of deep architectures.* In Neural Networks: Tricks of the Trade, Springer.
- **Bonferroni, C.E. (1936).** *Teoria statistica delle classi e calcolo delle probabilità.* Pubblicazioni del R Istituto Superiore di Scienze Economiche e Commerciali di Firenze.
- **Breiman, L. (2001).** *Random Forests.* Machine Learning, 45(1), 5–32.
- **Demšar, J. (2006).** *Statistical comparisons of classifiers over multiple data sets.* Journal of Machine Learning Research, 7, 1–30.
- **Ding, H., Trajcevski, G., Scheuermann, P., Wang, X., & Keogh, E. (2008).** *Querying and mining of time series data.* PVLDB, 1(2).
- **Dong, C., et al. (2015).** *Gesture recognition based on deep learning.* Pattern Recognition Letters.
- **Graves, A., & Schmidhuber, J. (2005).** *Framewise phoneme classification with bidirectional LSTM and other neural network architectures.* Neural Networks, 18(5–6).
- **Graves, A., Mohamed, A., & Hinton, G. (2013).** *Speech recognition with deep recurrent neural networks.* ICASSP 2013.
- **Huang, D., et al. (2019).** *3D gesture recognition dataset.* Source dataset utilisé dans ce projet.
- **Keogh, E., et al. (2005).** *HOT SAX: Efficiently finding the most unusual time series subsequence.* ICDM 2005.
- **Kingma, D.P., & Ba, J. (2014).** *Adam: A method for stochastic optimization.* ICLR 2015.
- **Núñez, J.C., et al. (2018).** *Convolutional neural networks and long short-term memory for skeleton-based human activity and hand gesture recognition.* Pattern Recognition, 76.
- **Probst, P., & Boulesteix, A.L. (2017).** *To tune or not to tune the number of trees in random forest.* JMLR, 18(1).
- **Wobbrock, J.O., et al. (2007).** *Gestures without libraries, toolkits or training: a $1 recognizer.* UIST 2007.
- **Xi, X., Keogh, E., Shelton, C., Wei, L., & Ratanamahatana, C.A. (2006).** *Fast time series classification using numerosity reduction.* ICML 2006.
- **Zhang, Z., et al. (2018).** *Improved Adam optimizer for deep neural networks.* IEEE Access.